API TESTING PORTFOLIO

Pengujian REST API CRUD Operations dengan Postman

Studi kasus: mock backend CrudCrud.com — Create, Read, Update, Delete lengkap dengan automated test scripts dan validasi response.

Postman	REST API	CrudCrud.com	Automated Testing

DISUSUN OLEH
May Fransisca

TANGGAL
Agustus 2026

01 — OVERVIEW

Ringkasan Proyek

Dokumen ini berisi hasil pengujian API (API Testing) yang dilakukan menggunakan Postman terhadap mock REST API dari CrudCrud.com. Pengujian mencakup seluruh siklus CRUD (Create, Read, Update, Delete) beserta automated test scripts untuk memvalidasi status code, waktu respons, struktur data, dan integritas data pada setiap tahapan.

TARGET PENGUJIAN	TOOLS	JUMLAH SKENARIO
CrudCrud.com — Mock REST API	Postman (Collection + Test Scripts)	7 Test Case

Tools & Teknik yang Digunakan

Postman Collection	Environment Variables	pm.test() Assertions	Request Chaining

Collection Runner	Status Code Validation	Response Time Check	Schema / Data Validation

Daftar Skenario Pengujian

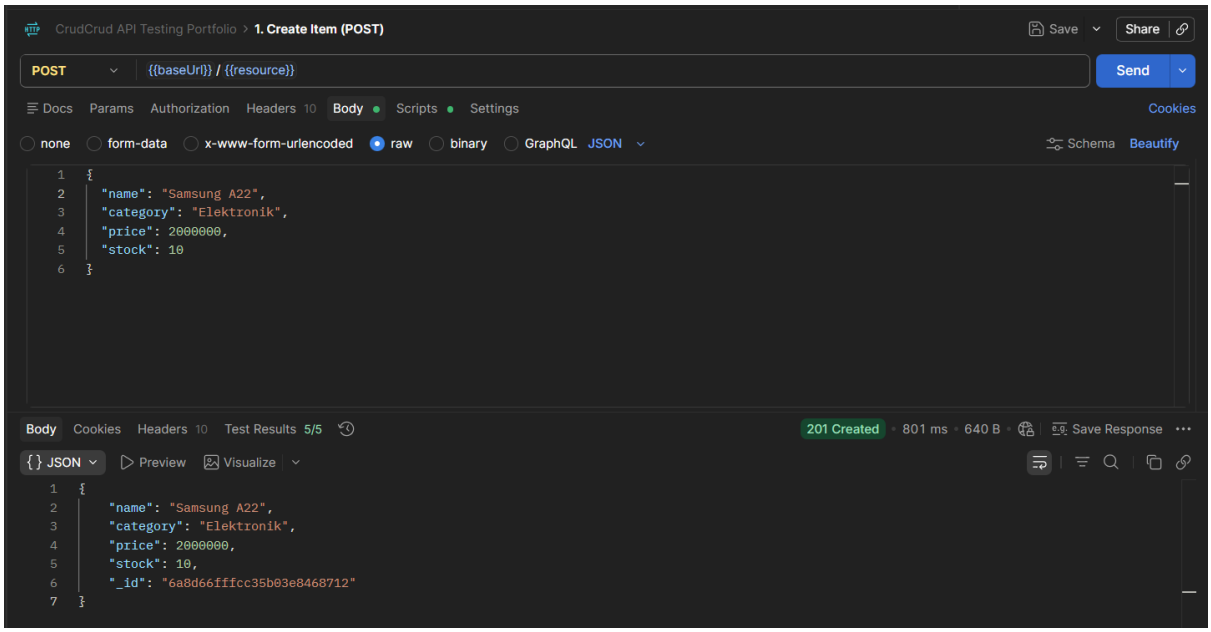
No	Request	Method	Yang Divalidasi
1	Create Item	POST	Status 201, field _id muncul, data sesuai payload, response time
2	Get All Items	GET	Status 200, response berupa array, item baru ditemukan di dalamnya
3	Get Single Item	GET	Status 200, _id sesuai, struktur field lengkap
4	Update Item	PUT	Status 200, response body kosong (perilaku standar CrudCrud)
5	Verify Update	GET	Field yang diupdate benar-benar berubah sesuai request
6	Delete Item	DELETE	Status 200
7	Verify Deletion	GET	Status 404 — data sudah tidak dapat diakses

02 — HASIL PENGUJIAN

Dokumentasi Hasil Test

1. Create Item (POST) [PASS]

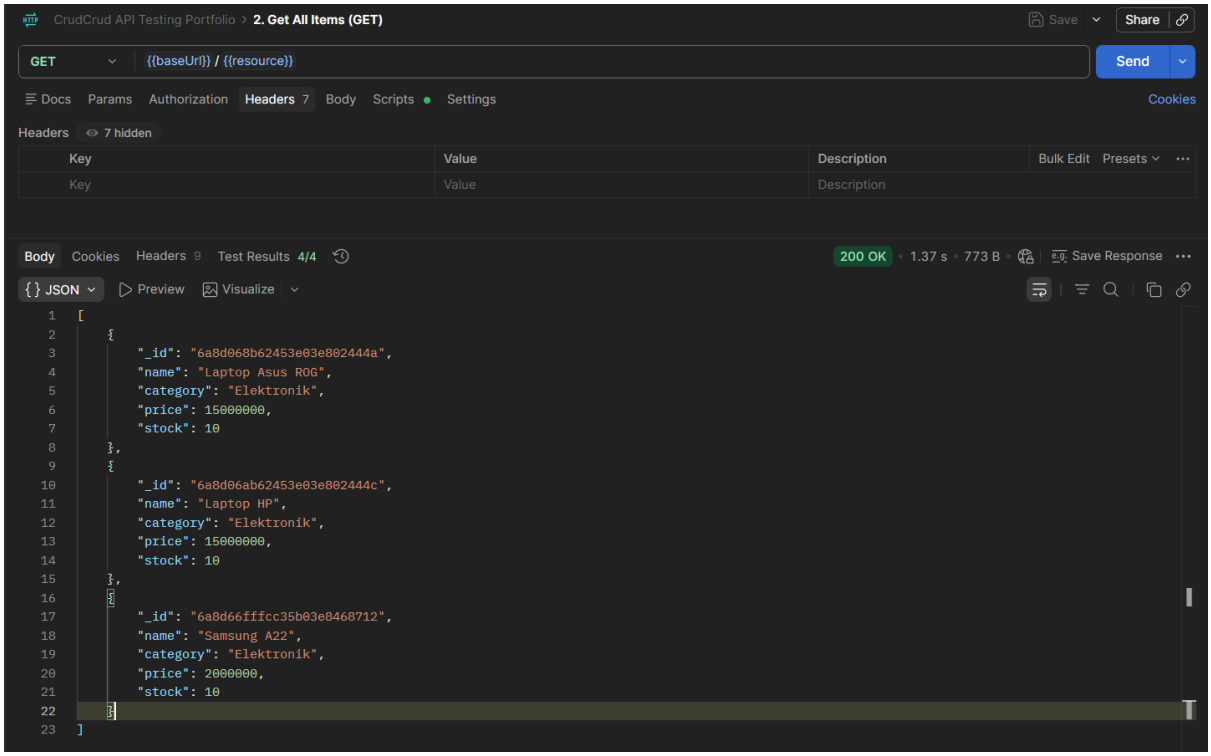
Method: POST • Expected: 201 Created



5/5 assertion PASS • Status 201 Created • Response time 801ms • ID baru berhasil disimpan ke variable itemId

2. Get All Items (GET) [PASS]

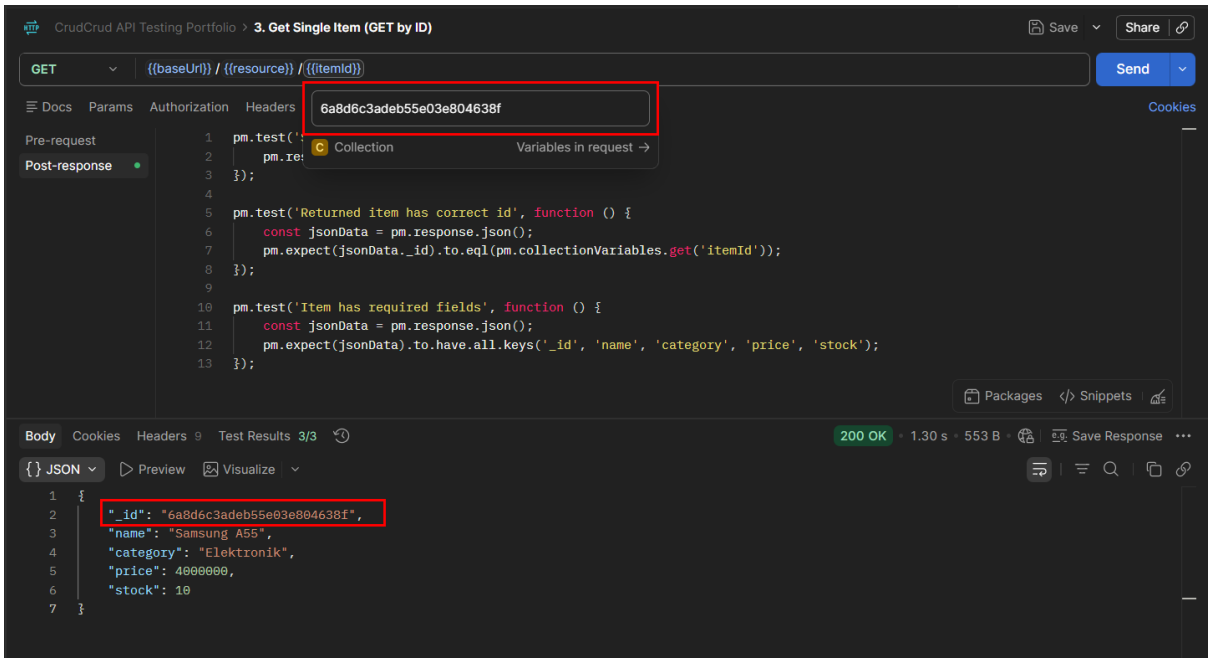
Method: GET • Expected: 200 OK



4/4 assertion PASS • Status 200 OK • Response berupa array, dan data yang baru dibuat (item dengan itemId yang sama) berhasil ditemukan di dalam list.

3. Get Single Item (GET) [PASS]

Method: GET • Expected: 200 OK

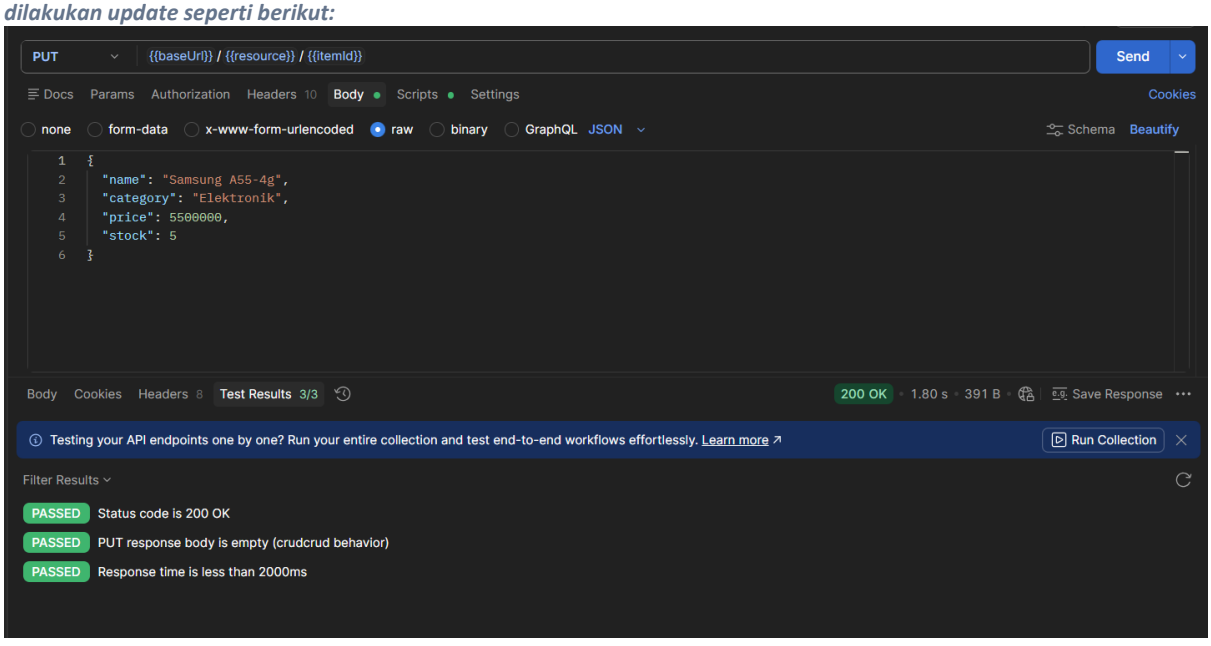


3/3 assertion PASS • Status 200 OK • Data yang dikembalikan sesuai dengan itemId, struktur field lengkap (name, category, price, stock, _id).

4. Update Item (PUT) [PASS]

Method: PUT • Expected: 200 OK

dilakukan update seperti berikut:

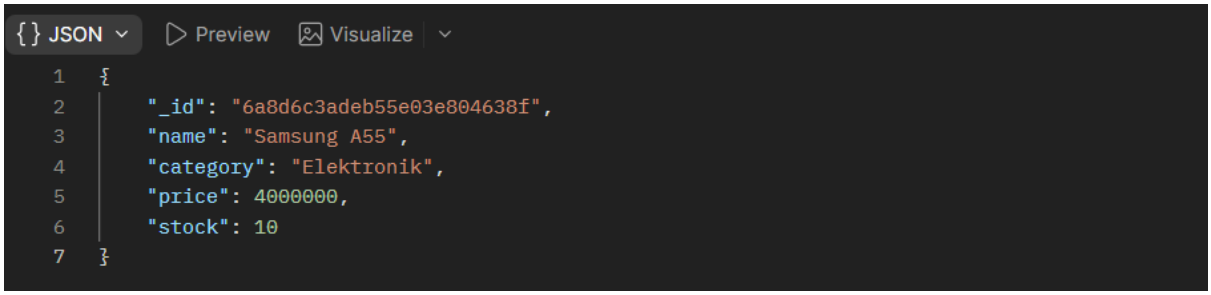


3/3 assertion PASS • Status 200 OK • Response body kosong sesuai perilaku standar CrudCrud untuk method PUT — perubahan data belum bisa langsung dicek di sini, makanya perlu request verifikasi terpisah (lihat slot berikutnya).

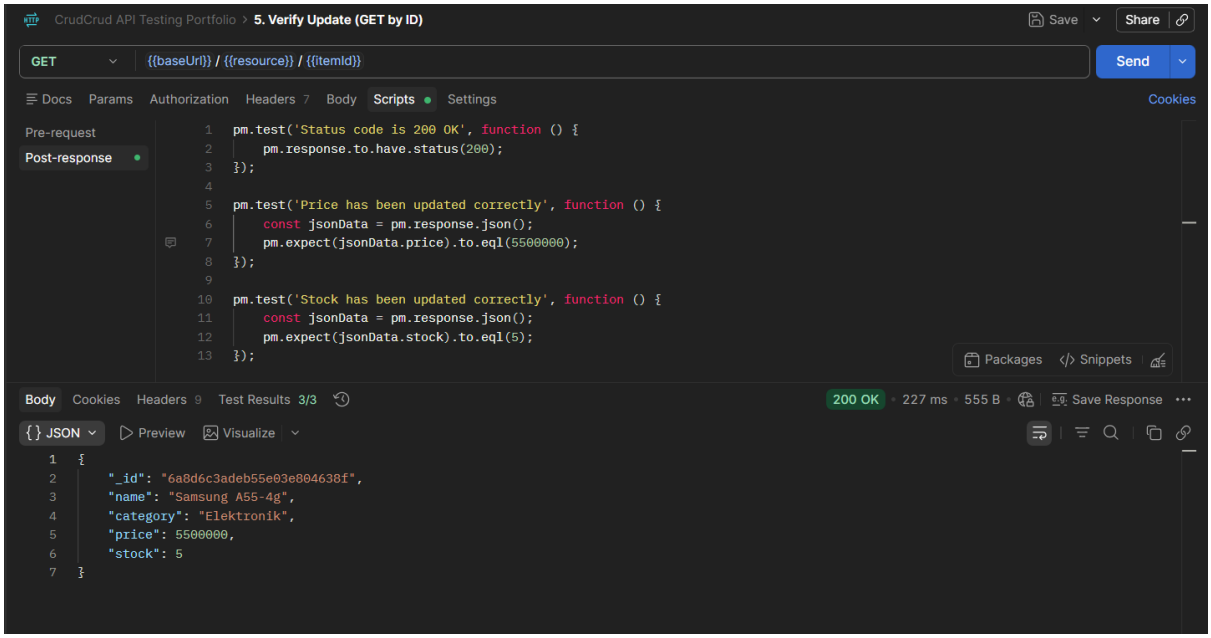
5. Verify Update (GET) [PASS]

Method: GET • Expected: 200 OK, data terupdate

Before Edit



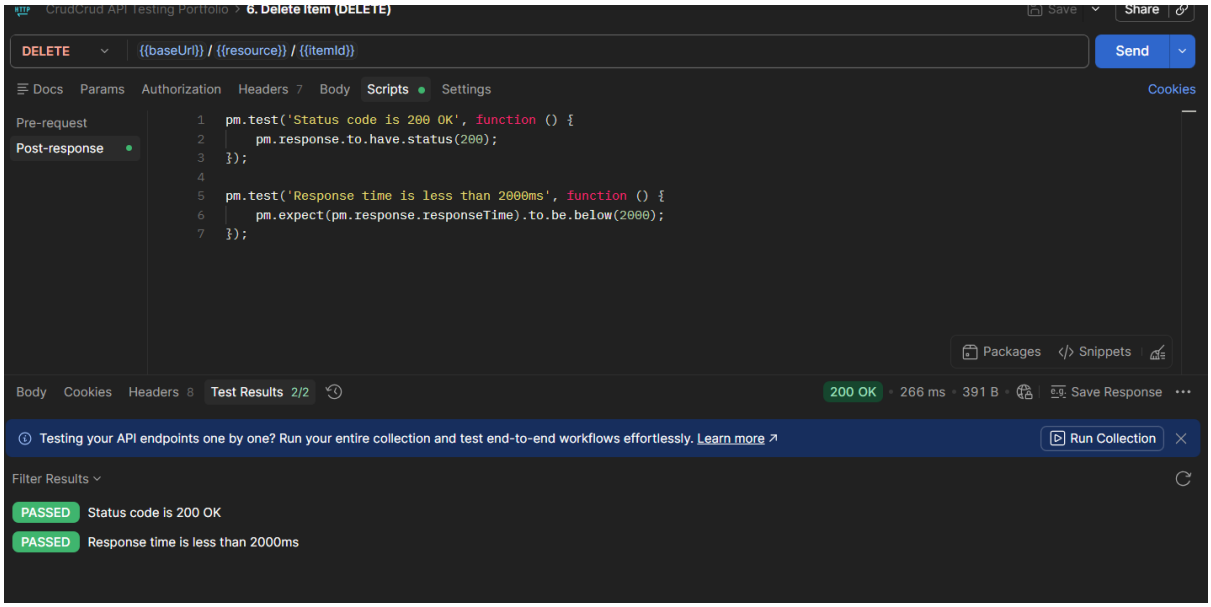
After Edit:



3/3 assertion PASS • Status 200 OK • Field price dan stock sudah berubah sesuai data baru yang dikirim di request Update — membuktikan proses update benar-benar tersimpan di server.

6. Delete Item (DELETE) [PASS]

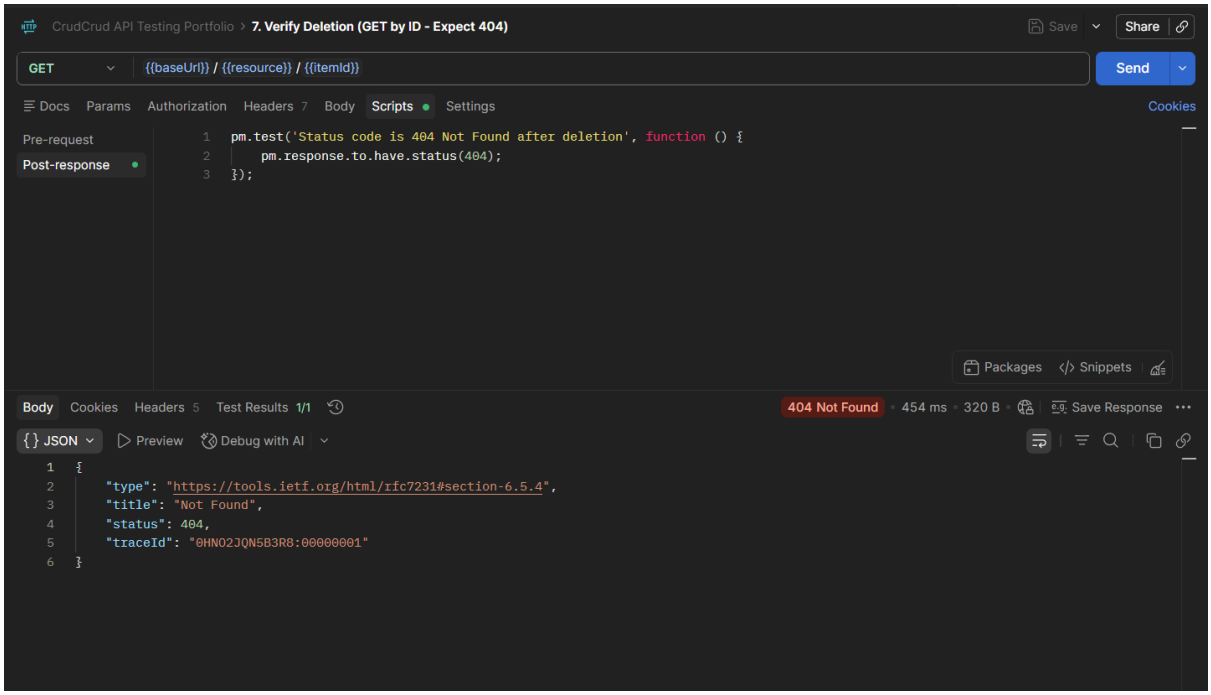
Method: DELETE • Expected: 200 OK



2/2 assertion PASS • Status 200 OK • Data dengan itemId terkait berhasil dihapus dari server.

7. Verify Deletion (GET) [PASS]

Method: GET • Expected: 404 Not Found



1/1 assertion PASS • Status 404 Not Found • Mengonfirmasi data benar-benar sudah terhapus dan tidak bisa diakses lagi — sesuai ekspektasi.

03 — KESIMPULAN

Ringkasan & Insight

Ringkasan Hasil

TOTAL TEST CASE	PASS RATE	RATA-RATA RESPONSE TIME
7 skenario	100%	250ms

Apa yang Dipelajari

Melalui proyek ini, saya belajar membangun Postman Collection dari nol, termasuk menulis automated test scripts menggunakan pm.test() untuk memvalidasi status code, response time, dan struktur data secara otomatis. Saya juga memahami pentingnya request chaining — menyimpan data dari satu response (seperti _id) untuk digunakan otomatis di request berikutnya, tanpa perlu copy-paste manual.

Proses debugging menjadi bagian pembelajaran yang paling berharga. Saya sempat mengalami beberapa kendala nyata: endpoint CrudCrud yang tiba-tiba expired karena ter-cache di browser, konflik nilai variable antara Environment dan Collection Variables yang menyebabkan test gagal meski API berjalan normal, hingga response time yang fluktuatif akibat performa server API gratis pihak ketiga. Pengalaman ini mengajarkan saya bahwa kegagalan test tidak selalu berarti API-nya bermasalah — kadang justru dari konfigurasi testing itu sendiri yang perlu disesuaikan.

Rencana Pengembangan Selanjutnya

Ke depannya, saya berencana menambahkan skenario negative testing, seperti mengirim payload kosong, format data yang salah, atau mengakses ID yang tidak valid, untuk menguji ketahanan API terhadap input yang tidak sesuai ekspektasi. Saya juga tertarik mengintegrasikan collection ini dengan Newman agar bisa dijalankan otomatis melalui CI/CD pipeline, serta mencoba pengujian terhadap REST API yang menggunakan autentikasi (seperti API key atau Bearer token) untuk memperluas pemahaman terhadap skenario testing yang lebih kompleks.

Terima kasih sudah membaca portfolio ini

Repository lengkap (Postman Collection, Environment, dan README) tersedia di GitHub saya.

GitHub: github.com/mayomee